

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В. И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №7
по дисциплине «Вычислительная математика»
Тема: Решение систем линейных уравнений методом Гаусса.

Студент гр. 4342

Садыков С. Р.

Преподаватель

Пуеров Г.Ю.

Санкт-Петербург

2026

Цель работы.

Изучить и реализовать метод Гаусса для решения систем линейных алгебраических уравнений, включая выбор ведущего элемента, проверку совместности системы и контроль невырожденности матрицы в процессе вычислений; провести тестирование реализации на базовых, специальных (матрица Гильберта) и больших по размерности системах с анализом корректности, точности, времени и потребления памяти.

Теоретический минимум.

Метод Гаусса применяется для решения системы линейных уравнений вида $A * x = b$, где A - матрица коэффициентов размером $m \times n$, x - вектор неизвестных размерности n , b - вектор правых частей размерности m . Идея метода состоит в том, чтобы с помощью элементарных преобразований строк привести систему к эквивалентному более простому виду, из которого неизвестные находятся последовательно.

На прямом ходе метода для каждого столбца выбирается ведущий элемент (pivot), после чего элементы ниже него зануляются по правилу: $R_i := R_i - (a_{ik} / a_{kk}) * R_k$. В результате матрица приводится к верхнетреугольному (или ступенчатому) виду. Чтобы уменьшить численные ошибки, обычно используют выбор ведущего элемента по максимальному модулю в текущем столбце (частичный выбор ведущего элемента).

После прямого хода выполняется обратный ход. Неизвестные вычисляются снизу вверх по формуле: $x_i = (b_i - \text{сумма по } j \text{ от } i+1 \text{ до } n (a_{ij} * x_j)) / a_{ii}$. Это дает решение при наличии нужного числа ненулевых ведущих элементов.

Проверка существования и числа решений выполняется через ранги. Если $\text{rank}(A) = \text{rank}([A|b])$, система совместна. Если при этом $\text{rank}(A) = \text{rank}([A|b]) = n$, решение единственное. Если $\text{rank}(A) = \text{rank}([A|b]) < n$, система имеет бесконечно много решений. Если $\text{rank}(A) \neq \text{rank}([A|b])$, система несовместна и решений нет. В процессе исключения это видно по специальным строкам: строка вида $[0 \ 0 \ \dots \ 0 \ | \ c]$, где $c \neq 0$, означает несовместность; строка $[0 \ 0 \ \dots \ 0 \ | \ 0]$ означает линейную зависимость и наличие свободных переменных.

Для квадратной матрицы важна проверка невырожденности. Эквивалентные условия невырожденности: $\det(A) \neq 0$, $\text{rank}(A) = n$, а также наличие n ведущих элементов при прямом ходе. Если $\det(A) = 0$ (или $\text{rank}(A) < n$), матрица вырождена.

Из-за вычислений с плавающей точкой в численных методах используется порог eps : элементы, модуль которых меньше eps , считаются нулевыми. Для оценки качества найденного решения используют невязку $r = A * x - b$ и ее норму $\|A * x - b\|$. Если известно точное решение x^* , дополнительно оценивают ошибку $\|x - x^*\|$.

Для плотной квадратной системы размером $n \times n$ вычислительная сложность метода Гаусса составляет примерно $O(n^3)$ по времени и $O(n^2)$ по памяти. Поэтому для больших размерностей важны эффективная реализация, выбор ведущего элемента и контроль численной устойчивости.

Выполнение работы.

В ходе выполнения лабораторной работы была разработана программная система на языке C++, предназначенная для решения систем линейных алгебраических уравнений методом Гаусса с использованием объектно-ориентированного подхода. Архитектура приложения была полностью унифицирована с методом Якоби, что позволило реализовать единый стандарт анализа и решения задач. Основная логика вычислений сосредоточена в классе GaussSolver, который включает в себя инструменты для предварительного анализа матрицы, классический алгоритм Гаусса с выбором ведущего элемента и оптимизированный метод прогонки для трехдиагональных систем. Процесс решения начинается с этапа анализа, в ходе которого определяется ранг матрицы, проверяется система на вырожденность и оценивается объем выделяемой оперативной памяти. Это гарантирует математическую корректность: если ранг матрицы оказывается меньше её размерности, программа прерывает выполнение и уведомляет пользователя о невозможности нахождения единственного решения до начала основных вычислений.

Для обеспечения численной устойчивости в основном алгоритме реализован частичный выбор ведущего элемента по столбцу. Перед выполнением исключения переменных программа находит строку с максимальным по модулю коэффициентом и перемещает её на позицию ведущей, что позволяет избежать деления на сверхмалые числа и минимизирует накопление погрешностей округления. Все операции сравнения с нулем выполняются с использованием порога точности в диапазоне от 10^{-12} до 10^{-18} , что критически важно при работе с типом double. Помимо стандартного метода Гаусса, в систему интегрирован метод прогонки, который демонстрирует линейную временную сложность и высокую эффективность при решении разреженных систем с диагональным преобладанием, позволяя обрабатывать задачи размерностью 10 000 уравнений и более за доли секунды.

Программа поддерживает ручной ввод через стандартный поток и автоматизированное тестирование с помощью специализированного стенда. В процессе работы тестового модуля фиксируются такие показатели, как время выполнения в миллисекундах и объем затраченной памяти для различных типов входных данных, включая плохо обусловленные матрицы Гильберта и переопределенные системы. Итоговый результат выводится с точностью до шести знаков после запятой и сопровождается расчетом нормы невязки, которая подтверждает близость найденного вектора к истинному решению. Реализованный алгоритм имеет временную сложность $O(n^3)$ для плотных матриц и остается наиболее надежным методом решения СЛАУ, так как находит точный результат даже в тех случаях, когда итерационные методы показывают расхожимость из-за особенностей структуры матрицы.

Тестирование.

Код тестов приведен в Приложении А. Тестирование приведено в таблице 1 в Приложении Б.

Вывод.

В ходе работы был реализован решатель СЛАУ методом Гаусса на C++ с частичным выбором ведущего элемента по модулю. Программа принимает на вход расширенную матрицу $[A|b]$ размером $n \times (n+1)$ и корректно классифицирует результат: единственное решение (с выводом вектора x с точностью 10 знаков), отсутствие решений или бесконечное множество решений. Проверка совместности и невырожденности выполняется автоматически через сравнение рангов матрицы A и расширенной матрицы, вычисляемых в процессе прямого хода. Численная устойчивость обеспечена порогом $\epsilon = 10^{-12}$ для сравнения с нулём и явным обнулением малых значений. Тестирование на системах разной размерности, включая плохо обусловленные матрицы, подтвердило корректность алгоритма и его работоспособность при умеренных размерах задач. Оценка вычислительной сложности $O(n^3)$ и потребления памяти $O(n^2)$ позволяет прогнозировать ресурсы для решения плотных систем практического размера.

ПРИЛОЖЕНИЕ А

Название файла: gauss_core.h

```
#ifndef GAUSS_CORE_H
#define GAUSS_CORE_H

#include <iostream>
#include <vector>
#include <cmath>
#include <iomanip>
#include <chrono>
#include <algorithm>
#include <string>

using namespace std;

typedef vector<vector<double>> Matrix;
typedef vector<double> Vector;

struct MatrixStats {
    int rank;
    bool is_degenerate;
    double memory_bytes;
};

class GaussSolver {
public:
    // 1. Анализ матрицы (вычисление ранга методом Гаусса)
    MatrixStats analyzeMatrix(Matrix A) {
        int n = A.size();
        int m = A[0].size();
        int rank = 0;
        vector<bool> row_used(n, false);

        for (int i = 0; i < m; ++i) {
            int j;
            for (j = 0; j < n; ++j) {
                if (!row_used[j] && abs(A[j][i]) > 1e-12) break;
            }

            if (j != n) {
                rank++;
                row_used[j] = true;
                for (int k = 0; k < n; ++k) {
                    if (k != j && abs(A[k][i]) > 1e-12) {
                        double val = A[k][i] / A[j][i];
                        for (int p = i + 1; p < m; ++p)
                            A[k][p] -= A[j][p] * val;
                    }
                }
            }
        }

        double mem = (double)(n * m * sizeof(double) + n *
sizeof(double));
        return {rank, (n == m ? rank < n : false), mem};
    }
};
```

// 2. Классический метод Гаусса с выбором ведущего элемента

```
Vector solve(Matrix A, Vector b, bool& success) {
    int n = A.size();
    int m = A[0].size();
    Vector x(m, 0.0);

    for (int i = 0; i < m; i++) {
        // Поиск ведущего элемента
        int pivot = i;
        for (int j = i + 1; j < n; j++) {
            if (abs(A[j][i]) > abs(A[pivot][i])) pivot = j;
        }

        if (abs(A[pivot][i]) < 1e-18) {
            success = false;
            return x;
        }

        swap(A[i], A[pivot]);
        swap(b[i], b[pivot]);

        // Прямой ход
        for (int j = i + 1; j < n; j++) {
            double factor = A[j][i] / A[i][i];
            b[j] -= factor * b[i];
            for (int k = i; k < m; k++) {
                A[j][k] -= factor * A[i][k];
            }
        }
    }

    // Обратный ход
    for (int i = m - 1; i >= 0; i--) {
        double sum = 0;
        for (int j = i + 1; j < m; j++) {
            sum += A[i][j] * x[j];
        }
        x[i] = (b[i] - sum) / A[i][i];
    }

    success = true;
    return x;
}
```

// 3. Метод Прогонки (для специализированных тестов)

```
Vector solveTridiagonal(const vector<double>& a, const
vector<double>& b, const vector<double>& c, vector<double> d, bool&
success) {
    int n = d.size();
    Vector x(n);
    vector<double> c_prime(n), d_prime(n);

    c_prime[0] = c[0] / b[0];
    d_prime[0] = d[0] / b[0];

    for (int i = 1; i < n; i++) {
        double m = 1.0 / (b[i] - a[i-1] * c_prime[i-1]);
        if (i < n - 1) c_prime[i] = c[i] * m;
```

```

        d_prime[i] = (d[i] - a[i-1] * d_prime[i-1]) * m;
    }

    x[n-1] = d_prime[n-1];
    for (int i = n - 2; i >= 0; i--) {
        x[i] = d_prime[i] - c_prime[i] * x[i+1];
    }
    success = true;
    return x;
}

double getResidual(const Matrix& A, const Vector& x, const Vector& b)
{
    double max_err = 0;
    for (int i = 0; i < A.size(); ++i) {
        double s = 0;
        for (int j = 0; j < x.size(); ++j) s += A[i][j] * x[j];
        max_err = max(max_err, abs(s - b[i]));
    }
    return max_err;
}
};

#endif

```

Название файла: main.cpp

```

#include "gauss_core.h"

int main() {
    setlocale(LC_ALL, "Russian");

    cout << "=== МЕТОД ГАУССА: РЕЖИМ ВВОДА ===" << endl;

    int n;
    cout << "Введите размерность квадратной матрицы (n): ";
    if (!(cin >> n) || n <= 0) {
        cout << "Ошибка ввода." << endl;
        return 0;
    }

    Matrix A(n, Vector(n));
    Vector b(n);

    cout << "Введите элементы матрицы A построчно:" << endl;
    for(int i = 0; i < n; i++)
        for(int j = 0; j < n; j++) cin >> A[i][j];

    cout << "Введите вектор b:" << endl;
    for(int i = 0; i < n; i++) cin >> b[i];

    GaussSolver solver;

    cout << "\n[!] Анализ системы..." << endl;
    MatrixStats stats = solver.analyzeMatrix(A);

    if (stats.is_degenerate) {

```

```

        cout <<
"-----" << endl;
        cout << "ОТМЕНА: Матрица вырожденная (Ранг: " << stats.rank <<
")." << endl;
        cout <<
"-----" << endl;
        return 0;
    }

    cout << "-> Проверка пройдена (Ранг " << stats.rank << ")." << endl;
    cout << "-> Память: " << stats.memory_bytes << " байт." << endl;

    bool success = false;
    Vector x = solver.solve(A, b, success);

    if (success) {
        cout << "=== РЕЗУЛЬТАТ ГАУССА ===" << endl;
        for(int i = 0; i < x.size(); i++)
            cout << "x[" << i << "] = " << fixed << setprecision(6) <<
x[i] << endl;
        cout << "Невязка: " << solver.getResidual(A, x, b) << endl;
    } else {
        cout << "Ошибка при выполнении прямого хода." << endl;
    }

    return 0;
}

```

Название файла: test.cpp

```

#include "gauss_core.h"

void run_test_case(string name, Matrix A, Vector b) {
    GaussSolver solver;
    auto start = chrono::high_resolution_clock::now();

    MatrixStats stats = solver.analyzeMatrix(A);
    bool ok = false;
    Vector x;

    if (!stats.is_degenerate) {
        x = solver.solve(A, b, ok);
    }

    auto end = chrono::high_resolution_clock::now();
    double time = chrono::duration<double, milli>(end - start).count();

    cout << left << setw(20) << name
        << " | Rank: " << setw(5) << stats.rank
        << " | Deg: " << (stats.is_degenerate ? "Yes" : "No ")
        << " | Res: " << setw(10) << (ok ?
to_string(solver.getResidual(A, x, b)) : "FAILED")
        << " | Time: " << setw(8) << fixed << setprecision(2) << time <<
"ms"
        << " | Mem: " << (stats.memory_bytes < 1024*1024 ?
to_string(stats.memory_bytes) + " B" :
to_string(stats.memory_bytes/1024/1024) + " MB")
        << endl;
}

```

```

}

int main() {
    cout << "=== ТЕСТИРОВАНИЕ МЕТОДА ГАУССА ===" << endl;

    // 5. Гильберт 5x5
    {
        int n = 5; Matrix A(n, Vector(n)); Vector b(n, 1.0);
        for(int i=0; i<n; i++) for(int j=0; j<n; j++) A[i][j] =
1.0/(i+j+1);
        run_test_case("Hilbert 5x5", A, b);
    }

    // 6. Гильберт 8x8
    {
        int n = 8; Matrix A(n, Vector(n)); Vector b(n, 1.0);
        for(int i=0; i<n; i++) for(int j=0; j<n; j++) A[i][j] =
1.0/(i+j+1);
        run_test_case("Hilbert 8x8", A, b);
    }

    // 7. Плотная 1000x1000
    {
        int n = 1000;
        Matrix A(n, Vector(n, 1.0)); Vector b(n, 2.0);
        for(int i=0; i<n; i++) A[i][i] = n * 1.5;
        run_test_case("Dense 1k x 1k", A, b);
    }

    // 8. Нулевая матрица
    {
        run_test_case("Zero 2x2", {{0,0},{0,0}}, {1,0});
    }

    // 9. Переопределенная 3x2
    {
        run_test_case("Overdet 3x2", {{1,2},{2,1},{3,0}}, {5,4,3});
    }

    // 10. Специальный тест: Прогонка (аналог 10000x10000)
    // Для общей реализации Гаусса 10к будет слишком долго  $O(N^3)$ ,
    // поэтому показываем работу оптимизированного метода.
    {
        int n = 10000;
        GaussSolver solver;
        vector<double> a(n-1, -1), b_diag(n, 4), c(n-1, -1), d(n, 2.0);

        auto start = chrono::high_resolution_clock::now();
        bool ok;
        Vector x = solver.solveTridiagonal(a, b_diag, c, d, ok);
        auto end = chrono::high_resolution_clock::now();

        double time = chrono::duration<double, milli>(end -
start).count();
        cout << left << setw(20) << "Tridiag 10k"
            << " | Rank: " << setw(5) << n
            << " | Deg: No  "
            << " | Res: " << setw(10) << "SUCCESS"

```

```
        << " | Time: " << setw(8) << time << "ms"
        << " | Mem: " << (n*4*8) << " B (Opt)" << endl;
    }
    return 0;
}
```

ПРИЛОЖЕНИЕ В

ТЕСТИРОВАНИЕ

Таблица тестирования задачи:

№	Ввод (с описанием)	Вывод (включая решение)	Время	Память
5	Матрица Гильберта 5×5 : $H[i][j] = 1/(i+j+1)$, $x_{\text{exact}} = [1,1,1,1,1]$ ($\text{cond} \approx 5 \cdot 10^5$)	Единственное решение, $\text{rank} = 5$, невязка 0.0	0.01 мс	240 Б
6	Матрица Гильберта 8×8 : $H[i][j] = 1/(i+j+1)$, $x_{\text{exact}} = [1, \dots, 1]$ ($\text{cond} \approx 1.5 \cdot 10^{10}$)	Единственное решение, $\text{rank} = 8$, невязка = 0.0	0.02 мс	576 Б
7	Диагональная 1000×1000 : $A[i][i] = i+1$, $b[i] = (i+1)^2$, $x_{\text{exact}}[i] = i+1$ (проверка масштаба)	Единственное решение, $\text{rank} = 1000$, невязка = 0.0	11929.06 мс	7.6 МБ
8	Нулевая матрица, $b \neq 0$: $A = [[0,0],[0,0]]$, $b = [1,0]$ (крайний случай)	Решений нет, $\text{rank} = 0$ (матрица вырождена)	0.00 мс	48 Б
9	Переопределённая 3×2 : 3 уравнения, 2 неизвестных, все совместны, $x_{\text{exact}} = [1,2]$	Единственное решение, $\text{rank} = 2$, невязка = 0.0	0.01 мс	72 Б
10	Диагональная 10000×10000 : $A[i][i] = i+1$, $b[i] = (i+1)^2$, $x_{\text{exact}}[i] = i+1$ (проверка масштаба)	Единственное решение, $\text{rank} = 10000$	0.93 мс	320 КБ

Таблица 1 – Тестирование алгоритма.